# ScrollSense

Assignment 2: Query Processing and Concurrency

Abhijeet Kumar
DA24B001
Indian Institute of Technology Madras

October 2026

**Abstract**

In this assignment, I investigated query execution plans, composite indexes, join algorithms, predicate pushdown, and transaction concurrency using the ScrollSense database. I scaled the dataset, examined SQLite execution plans before and after index creation, estimated the I/O cost of alternative join strategies, and rewrote a query to make predicate pushdown explicit. I also tested SQLite's writer-locking behaviour using two independent database connections.

The experiments show that appropriate indexes can change SQLite's access paths, while the query planner still uses temporary structures for operations such as grouping and ordering. The concurrency test confirms that `BEGIN IMMEDIATE` prevents a second connection from acquiring a writer reservation while the first connection holds it. Where comparable timing measurements or experimental outcomes were not recorded, I distinguish the available evidence from theoretical analysis rather than reporting unsupported results.

# Contents

**1 Experimental Setup** **3**
1.1 Environment . . . 3
1.2 Method . . . 3

**2 Task 1: Access Paths and Scan-Cost Estimate** **4**
2.1 F4: Like Retractions Within 60 Seconds . . . 4
2.1.1 Plan before index creation . . . 4
2.1.2 Plan after index creation . . . 4
2.2 F7: Session-Level Inference Cost . . . 4
2.2.1 Plan before index creation . . . 5
2.2.2 Plan after index creation . . . 5
2.3 F10: Rolling Watch Analysis . . . 5
2.4 F12: Completed Recommendation Impressions . . . 6
2.4.1 Plan before index creation . . . 6
2.4.2 Plan after index creation . . . 6
2.5 One Full-Scan Cost Estimate . . . 7
2.6 Summary of Access-Path Findings . . . 7

**3 Task 2: Composite-Index Experiments** **7**
3.1 Index 1: Signal Retractions . . . 7
3.2 Index 2: Session Date Range . . . 8
3.3 Index 3: View Completion Check . . . 8
3.4 Rejected Index . . . 8
3.5 Runtime Evidence and Limitations . . . 8

**4 Task 3: Join Strategy and Cost Model** **9**
4.1 Queries Joining High-Volume Tables . . . 9
4.2 Cardinality and Page Estimates . . . 9
4.3 Block Nested-Loop Join . . . 9
4.4 Sort-Merge Join . . . 10
4.5 Hash Join . . . 10
4.6 Three-Table Join Ordering . . . 10

**5 Task 4: Predicate Pushdown** **11**
5.1 Query and Selective Predicate . . . 11
5.2 Naive Relational-Algebra Plan . . . 11
5.3 Pushed-Down Relational-Algebra Plan . . . 11
5.4 Equivalence Rule and Limitation . . . 11
5.5 Original SQL . . . 11
5.6 Rewritten SQL . . . 12
5.7 Execution-Plan Evidence . . . 13

**6 Task 5: Concurrency, Schedules, and Isolation** **13**
6.1 A1 Deliverable G.3 (T2): Read Isolation . . . 13
6.2 Proposed Write-Skew Scenario . . . 14
6.3 Two-Connection Test of `BEGIN IMMEDIATE` . . . 15
6.4 Prevention and Trade-Off . . . 15

**7 Conclusion** **15**

# 1 Experimental Setup

## 1.1 Environment

I carried out the experiments on a MacBook Air using Python and SQLite. I kept the original Assignment 1 files separate from the working files for Assignment 2 so that the original schema and queries remained available for reference.

| Item | Configuration |
|---|---|
| Database engine | SQLite 3.54.0 |
| Python | 3.14.8 in a virtual environment |
| Operating system | macOS |
| Database | `scrollsense_a2.db` |
| Scale factor | 50× |
| Assumed page size for cost model | 4096 bytes |
| Assumed average row size | 64 bytes |

Table 1: Experimental environment and cost-model assumptions.

I generated a scaled database with a scale factor of 50. The resulting database passed SQLite's integrity check. The following row counts were recorded after generation.

| Table | Recorded rows |
|---|---:|
| User | 250,000 |
| Video | 1,000,000 |
| Impression | 15,000,000 |
| ViewSegment | 8,892,547 |
| Signal | 562,275 |
| Session | 100,000 |
| Turn | 276,011 |
| ToolCall | 218,196 |

Table 2: Selected row counts in the scaled database.

I also made a small correction to the data generator so that generated phone numbers remain unique at the larger scale. I kept this change in the Assignment 2 working copy rather than modifying the original Assignment 1 version.

## 1.2 Method

For query-plan analysis, I used SQLite's `EXPLAIN QUERY PLAN`. I compared the recorded plans before and after creating the indexes in `indexes.sql`. I interpreted operators such as `SCAN`, `SEARCH`, `USING COVERING INDEX`, and `USE TEMP B-TREE` to understand how SQLite accessed and processed the data.

The page-transfer calculations in this report use a simplified textbook model. They are estimates, not measurements of physical disk I/O. Similarly, an execution plan establishes the access path chosen by SQLite but does not, by itself, establish a runtime improvement.

# 2 Task 1: Access Paths and Scan-Cost Estimate

## 2.1 F4: Like Retractions Within 60 Seconds

F4 finds cases where a user liked a video and subsequently retracted that like within 60 seconds. The query joins `Signal` to itself. The first alias represents the original like, and the second alias represents the retraction.

The relevant conditions are:

```
WHERE l.signal_type = 'like'
  AND r.signal_type = 'like_retraction'
  AND r.created_at > l.created_at
  AND (julianday(r.created_at) -
       julianday(l.created_at)) * 86400 <= 60
```

The query also orders the result by the original like timestamp and signal ID.

### 2.1.1 Plan before index creation

```
|--SCAN r
|--SEARCH l USING INTEGER PRIMARY KEY (rowid=?)
`--USE TEMP B-TREE FOR ORDER BY
```

The plan scans the retraction-side alias `r`. For each candidate, SQLite searches for the corresponding original signal using its integer primary key. A temporary B-tree is used to satisfy the ordering.

The full scan is important because `Signal` contains 562,275 rows in the scaled database. The plan does not mean that every pair of signals is compared; the primary-key lookup restricts the lookup for the matching original signal.

### 2.1.2 Plan after index creation

I created the following index:

```
CREATE INDEX IF NOT EXISTS
    idx_signal_type_retracts_created
ON Signal(signal_type, retracts_signal_id, created_at);
```

The recorded plan became:

```
|--SEARCH r USING COVERING INDEX
|  idx_signal_type_retracts_created (signal_type=?)
|--SEARCH l USING INTEGER PRIMARY KEY (rowid=?)
`--USE TEMP B-TREE FOR ORDER BY
```

SQLite now searches the index for the requested signal type rather than scanning the entire retraction-side table. The index also contains the referenced signal ID and timestamp, which makes it covering for the columns needed from the `r` alias. The sort remains because the index does not directly provide the complete requested ordering.

The F4 query took 0.09 seconds before the index was created and 0.03 seconds after it was recreated. This is a single-run observation rather than a controlled benchmark, so cache state and system load may affect the measured difference.

## 2.2 F7: Session-Level Inference Cost

F7 calculates the cost of each agent session, grouped by prompt template version. It joins `Session`, `Turn`, `PromptTemplateVersion`, and `ModelPricingPeriod`. The query filters sessions

to August 2026, aggregates token-based costs, applies a `HAVING` condition, and sorts by session cost.

The date predicate is:

```
WHERE s.started_at >= '2026-08-01T00:00:00Z'
  AND s.started_at <  '2026-09-01T00:00:00Z'
```

### 2.2.1 Plan before index creation

```
|--SCAN t
|--SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH ptv USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH p USING INDEX
|  sqlite_autoindex_ModelPricingPeriod_1
|  (model_id=? AND valid_from<?)
|--USE TEMP B-TREE FOR GROUP BY
`--USE TEMP B-TREE FOR ORDER BY
```

The original plan starts by scanning `Turn`. It then looks up the associated session, template version, and pricing period. SQLite uses temporary B-trees for grouping and ordering.

### 2.2.2 Plan after index creation

I created an index on the session start time and session ID:

```
CREATE INDEX IF NOT EXISTS
    idx_session_started_session
ON Session(started_at, session_id);
```

The recorded plan became:

```
|--SEARCH s USING COVERING INDEX
|  idx_session_started_session
|  (started_at>? AND started_at<?)
|--SEARCH t USING INDEX sqlite_autoindex_Turn_1
|  (session_id=?)
|--SEARCH ptv USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH p USING INDEX
|  sqlite_autoindex_ModelPricingPeriod_1
|  (model_id=? AND valid_from<?)
|--USE TEMP B-TREE FOR GROUP BY
`--USE TEMP B-TREE FOR ORDER BY
```

The planner now begins with the date-filtered sessions and then looks up matching turns. The leading `started_at` column supports the range predicate. The remaining joins and temporary grouping and sorting structures are still required.

The full F7 query took 0.13 seconds before the target index was created and 0.08 seconds after it was recreated. These are single-run wall-clock measurements; cache state and system load may affect the observed difference.

## 2.3 F10: Rolling Watch Analysis

F10 joins the high-volume `ViewSegment` and `Impression` tables to analyse rolling watch behaviour. The captured plan was:

```
|--CO-ROUTINE rolling_watch
|  |--MATERIALIZE daily_watch
|  |  |--SCAN vs
|  |  |--SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
```

```
|  |  `--USE TEMP B-TREE FOR GROUP BY
|  |--SCAN d1
|  |--BLOOM FILTER ON d2 (user_id=?)
|  |--SEARCH d2 USING AUTOMATIC COVERING INDEX
|  |  (user_id=?)
|  `--USE TEMP B-TREE FOR GROUP BY
|--SCAN rolling_watch
`--USE TEMP B-TREE FOR ORDER BY
```

The plan materializes the daily-watch intermediate result. It scans `ViewSegment`, searches `Impression` by integer primary key, and uses temporary B-trees for grouping and ordering. The self-join of the intermediate result uses a Bloom filter and an automatically created covering index on `user_id`.

## 2.4 F12: Completed Recommendation Impressions

F12 finds sessions in which a recommended video was shown and watched to completion. It joins `Session`, `Turn`, `AgentRecommendation`, and `Impression`, and uses a correlated `EXISTS` query against `ViewSegment`.

### 2.4.1 Plan before index creation

```
|--SCAN i
|--SEARCH ar USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
|--BLOOM FILTER ON vs (impression_id=? AND reached_end=?)
|--SEARCH vs EXISTS USING AUTOMATIC PARTIAL COVERING INDEX
|  (impression_id=? AND reached_end=?)
|--USE TEMP B-TREE FOR DISTINCT
`--USE TEMP B-TREE FOR ORDER BY
```

SQLite scans `Impression`, performs primary-key lookups in the related tables, and uses an automatically created partial covering index for the correlated completion check. Temporary B-trees support `DISTINCT` and ordering.

### 2.4.2 Plan after index creation

I created the following index:

```
CREATE INDEX IF NOT EXISTS
    idx_viewsegment_impression_reached
ON ViewSegment(impression_id, reached_end);
```

The resulting plan was:

```
|--SCAN i
|--SEARCH ar USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH vs EXISTS USING COVERING INDEX
|  idx_viewsegment_impression_reached
|  (impression_id=? AND reached_end=?)
|--SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
|--SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
|--USE TEMP B-TREE FOR DISTINCT
`--USE TEMP B-TREE FOR ORDER BY
```

The persistent covering index is used for the correlated existence check. The scan of `Impression` and the temporary structures for duplicate elimination and ordering remain.

### 2.5 One Full-Scan Cost Estimate

For the required scan-cost estimate, I use the recorded full scan of `ViewSegment` in F10.

The table contains:

$$N = 8{,}892{,}547 \text{ rows.}$$

For a simplified model, I assume a 4096-byte page and an average row size of 64 bytes. Therefore, the estimated number of rows per page is:

$$\text{Rows per page} = \left\lfloor \frac{4096}{64} \right\rfloor = 64.$$

The estimated number of pages is:

$$B(\text{ViewSegment}) = \left\lceil \frac{8{,}892{,}547}{64} \right\rceil = 138{,}946.$$

Under these assumptions, a sequential scan reads approximately 138,946 data pages. This is a simplified page-transfer estimate. Actual SQLite I/O depends on its page layout, record sizes, indexes, cache state, and other implementation details.

### 2.6 Summary of Access-Path Findings

| Query | Before | After |
|---|---|---|
| F4 | Scan of retraction-side `Signal`; primary-key lookup for original signal | Indexed search on signal type; primary-key lookup retained |
| F7 | Scan of `Turn`; session lookup by primary key | Date-range search on `Session`; indexed lookup into `Turn` |
| F10 | Full scan of `ViewSegment`; lookup into `Impression` | Plan inspected for high-volume join and aggregation |
| F12 | Scan of `Impression`; automatic partial covering index for completion check | Persistent covering index for the correlated completion check |

Table 3: Summary of recorded access paths.

## 3 Task 2: Composite-Index Experiments

### 3.1 Index 1: Signal Retractions

The first index is:

```
CREATE INDEX IF NOT EXISTS
    idx_signal_type_retracts_created
ON Signal(signal_type, retracts_signal_id, created_at);
```

The leading column, `signal_type`, supports the equality conditions that identify likes and retractions. The next column, `retracts_signal_id`, stores the reference to the signal being retracted. The timestamp is also stored in the index.

The recorded plan changed from `SCAN r` to `SEARCH r USING COVERING INDEX`. The plan still contains a temporary B-tree for ordering, so the index does not eliminate all query-processing work.

### 3.2 Index 2: Session Date Range

The second index is:

```
CREATE INDEX IF NOT EXISTS
    idx_session_started_session
ON Session(started_at, session_id);
```

The leading `started_at` column supports the date-range predicate in F7. The `session_id` column is included so that SQLite can obtain the session identifier directly from the index. The recorded plan uses a covering index for the range search.

The plan also changes the join sequence: SQLite starts with sessions in the selected date range and then searches for their turns. This is consistent with reducing the set of sessions before processing subsequent joins.

### 3.3 Index 3: View Completion Check

The third index is:

```
CREATE INDEX IF NOT EXISTS
    idx_viewsegment_impression_reached
ON ViewSegment(impression_id, reached_end);
```

F12 checks whether an impression has a `ViewSegment` row with `reached_end = 1`. Both columns in the correlated condition are included in the index. The recorded plan uses this persistent covering index instead of the automatically created partial covering index that appeared in the baseline plan.

### 3.4 Rejected Index

I considered but did not build the following index:

```
-- Not created:
-- CREATE INDEX idx_impression_recommendation_video_user
-- ON Impression(recommendation_id, video_id, user_id);
```

This index could support the matching conditions between `Impression` and `AgentRecommendation` in F12. However, `Impression` contains 15 million rows. Building another index on this table would require additional storage and index-build time. I left it uncreated rather than claiming a performance improvement that I did not measure.

### 3.5 Runtime Evidence and Limitations

The before-and-after execution plans were captured for the three index experiments. F4 changed from a scan of the retraction-side `Signal` alias to an indexed search. F7 changed from a scan of `Turn` to an indexed date-range search on `Session`. F12 changed from an automatically created partial covering index to the persistent covering index for the completion check.

The complete queries were timed once with the target index absent and once after recreating it. The measured wall-clock times were:

| Query | Before (s) | After (s) | Observed reduction (%) |
|---|---|---|---|
| F4: Signal retractions | 0.09 | 0.03 | 66.7 |
| F7: Session cost | 0.13 | 0.08 | 38.5 |
| F12: Completed recommendations | 6.94 | 3.46 | 50.1 |

The observed reduction is calculated as $(T_{\text{before}} - T_{\text{after}})/T_{\text{before}} \times 100$. These are single-run measurements, not controlled benchmarks. Cache state and system load may affect the results, so the reductions are preliminary observations rather than guaranteed performance improvements. The execution plans provide complementary evidence of changes in access paths.

# 4 Task 3: Join Strategy and Cost Model

## 4.1 Queries Joining High-Volume Tables

I used F10 and F12 to examine joins involving the large `ViewSegment` and `Impression` tables. F10 aggregates watch information across these tables. F12 matches impressions to recommendations and checks completion through `ViewSegment`.

The recorded plans contain scans, indexed searches, primary-key lookups, and temporary structures. SQLite uses nested-loop-style join processing rather than a native hash join or sort-merge join. In these plans, one input is scanned or searched and matching rows are retrieved from the other input through indexes or primary-key lookups.

This distinction matters because the cost of a nested-loop join depends heavily on the access path for the inner input. An indexed lookup is not equivalent to repeatedly scanning the entire inner table.

## 4.2 Cardinality and Page Estimates

For the simplified cost comparison, I use the recorded row counts and assume 64 rows per page:

$$B(R) = \left\lceil \frac{8{,}892{,}547}{64} \right\rceil = 138{,}946$$

for `ViewSegment`, and

$$B(S) = \left\lceil \frac{15{,}000{,}000}{64} \right\rceil = 234{,}375$$

for `Impression`.

These estimates use the same simplified page-size assumptions as Task 1. They are not the actual number of SQLite pages reported by the database.

## 4.3 Block Nested-Loop Join

Let $M = 100$ be the assumed number of buffer pages. The textbook block nested-loop join estimate is:

$$B(R) + \left\lceil \frac{B(R)}{M-2} \right\rceil B(S).$$

Substituting the estimates gives:

$$138{,}946 + \left\lceil \frac{138{,}946}{98} \right\rceil (234{,}375).$$

Since

$$\left\lceil \frac{138{,}946}{98} \right\rceil = 1{,}418,$$

the estimated number of page transfers is:

$$138{,}946 + 1{,}418(234{,}375) = 332{,}482{,}696.$$

This is the cost of the simplified block nested-loop model under the stated assumptions. It is not a measured cost of the actual SQLite execution.

### 4.4 Sort-Merge Join

Under the simplified two-pass model, a sort-merge join can be estimated at approximately:

$$3(B(R) + B(S)).$$

Therefore:

$$3(138{,}946 + 234{,}375) = 1{,}119{,}963$$

page transfers.

This estimate assumes that the inputs can be sorted in the available memory and that the relevant two-pass assumptions hold. If the data do not fit those assumptions, additional passes may be required.

### 4.5 Hash Join

Under the same simplified two-pass assumptions, a Grace hash join has an approximate cost of:

$$3(B(R) + B(S)) = 1{,}119{,}963$$

page transfers.

This estimate assumes that the partitions can be processed within the available memory and that the hash-join algorithm's assumptions hold. It is a theoretical comparison, not a claim that SQLite used a hash join.

| **Join model** | **Estimated page transfers** |
|---|---:|
| Block nested-loop | 332,482,696 |
| Two-pass sort-merge | 1,119,963 |
| Two-pass Grace hash | 1,119,963 |

Table 4: Theoretical cost comparison under the stated assumptions.

Under this simplified model, sort-merge and hash join have much lower estimated transfer costs than block nested-loop join for these input sizes. However, SQLite's actual plans use the join strategies available in SQLite and the indexes present in the database. The textbook estimates do not prove that the actual SQLite plan is more expensive than an unavailable algorithm would be in practice.

### 4.6 Three-Table Join Ordering

For a query involving `Session`, `Turn`, and `PromptTemplateVersion`, a reasonable join order is to filter `Session` by the date range first, then join the remaining sessions to `Turn`, and finally look up the corresponding template versions.

The reasoning is that the date condition can reduce the number of session rows before they are joined to the larger turn relation. The recorded F7 plan after index creation follows this general strategy: it searches the session date range, searches turns by session ID, and looks up template versions by integer primary key.

This is a plausible join order based on the predicate and the available indexes. A definitive claim that it is the cheapest possible order would require estimated intermediate cardinalities

for competing orders or measured comparisons. Those candidate intermediate-result estimates were not recorded, so I do not claim to have proved a globally cheapest order.

# 5 Task 4: Predicate Pushdown

## 5.1 Query and Selective Predicate

I used F7, which calculates session-level inference costs for sessions that started during August 2026. Its selective predicate restricts the session relation to the desired date interval:

```
s.started_at >= '2026-08-01T00:00:00Z'
AND s.started_at < '2026-09-01T00:00:00Z'
```

The predicate refers only to the `Session` relation. This makes it suitable for an explicit predicate-pushdown rewrite under the query's inner-join semantics.

## 5.2 Naive Relational-Algebra Plan

Let $S$, $T$, $V$, and $P$ denote `Session`, `Turn`, `PromptTemplateVersion`, and `ModelPricingPeriod`, respectively. Let $\sigma_d$ denote the selection for the August date range.

A naive logical plan can be represented as:

$$\sigma_d(S \bowtie T \bowtie V \bowtie P).$$

This places the selection after the joins in the logical expression. If implemented literally, it can produce intermediate join results containing sessions that are subsequently discarded by the date predicate.

## 5.3 Pushed-Down Relational-Algebra Plan

The pushed-down expression is:

$$(\sigma_d(S)) \bowtie T \bowtie V \bowtie P.$$

This applies the date predicate to `Session` before the joins. The intended benefit is to reduce the number of session tuples passed into the subsequent join operations.

## 5.4 Equivalence Rule and Limitation

For inner joins, a selection predicate that refers only to attributes of one input relation can be pushed below the join without changing the result, provided the join conditions and predicate semantics remain unchanged.

This rule must be applied carefully to outer joins. Moving a predicate across an outer join can change the result, particularly when the predicate filters out NULL-extended rows that the outer join was meant to preserve.

## 5.5 Original SQL

```
SELECT
    s.session_id,
    ptv.template_version_id,
    ptv.version_number,
    ROUND(
        SUM(
            (t.input_tokens - t.cached_input_tokens) * p.input_rate
```

```
            + t.cached_input_tokens * p.cached_input_rate
            + t.output_tokens * p.output_rate
        ),
        6
    ) AS session_cost
FROM Session s
JOIN Turn t
    ON t.session_id = s.session_id
JOIN PromptTemplateVersion ptv
    ON ptv.template_version_id =
       t.prompt_template_version_id
JOIN ModelPricingPeriod p
    ON p.model_id = t.model_id
   AND t.created_at >= p.valid_from
   AND (p.valid_to IS NULL OR t.created_at < p.valid_to)
WHERE s.started_at >= '2026-08-01T00:00:00Z'
  AND s.started_at < '2026-09-01T00:00:00Z'
GROUP BY
    s.session_id,
    ptv.template_version_id,
    ptv.version_number
HAVING session_cost > 0.05
ORDER BY
    session_cost DESC,
    s.session_id,
    ptv.version_number;
```

### 5.6 Rewritten SQL

I made the early filtering explicit using a common table expression named `filtered_sessions`:

```
WITH filtered_sessions AS (
    SELECT session_id, user_id
    FROM Session
    WHERE started_at >= '2026-08-01T00:00:00Z'
      AND started_at < '2026-09-01T00:00:00Z'
)
SELECT
    s.session_id,
    ptv.template_version_id,
    ptv.version_number,
    ROUND(
        SUM(
            (t.input_tokens - t.cached_input_tokens) * p.input_rate
            + t.cached_input_tokens * p.cached_input_rate
            + t.output_tokens * p.output_rate
        ),
        6
    ) AS session_cost
FROM filtered_sessions s
JOIN Turn t
    ON t.session_id = s.session_id
JOIN PromptTemplateVersion ptv
    ON ptv.template_version_id =
       t.prompt_template_version_id
JOIN ModelPricingPeriod p
    ON p.model_id = t.model_id
   AND t.created_at >= p.valid_from
```

```
   AND (p.valid_to IS NULL OR t.created_at < p.valid_to)
GROUP BY
    s.session_id,
    ptv.template_version_id,
    ptv.version_number
HAVING session_cost > 0.05
ORDER BY
    session_cost DESC,
    s.session_id,
    ptv.version_number;
```

Both versions retain the same date range, join conditions, cost calculation, grouping, `HAVING` condition, and ordering. The rewritten query makes the early selection explicit.

### 5.7 Execution-Plan Evidence

The recorded F7 plan after index creation includes:

```
SEARCH s USING COVERING INDEX idx_session_started_session
    (started_at>? AND started_at<?)
SEARCH t USING INDEX sqlite_autoindex_Turn_1 (session_id=?)
SEARCH ptv USING INTEGER PRIMARY KEY (rowid=?)
SEARCH p USING INDEX sqlite_autoindex_ModelPricingPeriod_1
    (model_id=? AND valid_from<?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

The range search on `Session` demonstrates that SQLite uses the date index for the original query. This is consistent with applying the selective predicate early. It does not, by itself, prove that the explicit common-table-expression rewrite is faster: SQLite may already perform an equivalent optimization on the original query.

The original F7 query took 0.08 seconds in the recorded after-index run. A directly comparable timing experiment between the original SQL and the explicit CTE rewrite was not performed. Therefore, the index timing does not establish an additional performance benefit from predicate pushdown.

## 6 Task 5: Concurrency, Schedules, and Isolation

### 6.1 A1 Deliverable G.3 (T2): Read Isolation

In the A1 T2 demonstration, Connection 1 began a transaction and inserted an uncommitted moderation decision for video 1 with state `live`. Connection 1 then queried the current state of the video and could observe its own uncommitted change. Before Connection 1 committed, Connection 2 queried the same video and observed the previously committed `pending` state. Connection 1 subsequently rolled back.

The relevant operation sequence is:

| Order | Connection | Operation |
|---|---|---|
| 1 | C1 | Begin transaction. |
| 2 | C1 | Insert an uncommitted `live` moderation decision for video 1. |
| 3 | C1 | Read the current state and observe `live`. |
| 4 | C2 | Read the current state and observe the committed `pending` state. |
| 5 | C1 | Roll back the transaction. |

Table 5: Summary of the A1 T2 read-isolation experiment.

This experiment demonstrates that Connection 2 does not see Connection 1's uncommitted insert. Because Connection 1 rolls back, the experiment does not establish a precedence edge between two committed transactions that would demonstrate a non-serializable schedule.

The corresponding precedence graph for the committed operations has no conflict edge between these two transactions. It is therefore acyclic for this demonstrated interleaving. This is a read-isolation experiment rather than evidence of a committed write-skew anomaly.

### 6.2 Proposed Write-Skew Scenario

I considered an application invariant involving videos 1 and 2: at least one of these videos must remain in the `live` state.

The proposed scenario has two transactions. Each reads the latest moderation state of both videos. Transaction $T_1$ then decides to update video 1, while transaction $T_2$ decides to update video 2. If both transactions base their decisions on the same original state, each may assume that the other video will remain live.

The abstract schedule is:

| Order | Transaction | Operation |
|---|---|---|
| 1 | $T_1$ | Read state of video 1. |
| 2 | $T_2$ | Read state of video 1. |
| 3 | $T_1$ | Read state of video 2. |
| 4 | $T_2$ | Read state of video 2. |
| 5 | $T_1$ | Write a new moderation decision for video 1. |
| 6 | $T_2$ | Write a new moderation decision for video 2. |

Table 6: Abstract schedule for the proposed write-skew scenario.

The conflicting operations yield two edges in the precedence graph:

- $T_2 \to T_1$: $T_2$'s read of video 1 occurs before $T_1$'s write to video 1.
- $T_1 \to T_2$: $T_1$'s read of video 2 occurs before $T_2$'s write to video 2.

The graph contains a cycle:

$$T_1 \to T_2 \to T_1.$$

Therefore, the abstract schedule is not conflict-serializable. If both transactions committed their independent updates, both videos could be demoted, violating the assumed invariant. This would be a write-skew anomaly.

However, the schedule above is an abstract concurrency analysis. It is not evidence that SQLite committed both writes. SQLite allows only one writer at a time, and in WAL mode a transaction that tries to write from an outdated read snapshot may fail with `SQLITE_BUSY_SNAPSHOT`. I did not record a successful committed execution of this anomaly.

### 6.3 Two-Connection Test of `BEGIN IMMEDIATE`

I tested SQLite's writer reservation using two independent Python SQLite connections against a disposable copy of the scaled database, named `concurrency_test.db`. Both connections used a one-second timeout.

Connection 1 executed `BEGIN IMMEDIATE` and acquired the writer reservation. While Connection 1 held it, Connection 2 attempted `BEGIN IMMEDIATE`. The observed output was:

```
Connection 1: BEGIN IMMEDIATE
Connection 1: writer reservation acquired
Connection 2: attempting BEGIN IMMEDIATE
Connection 2 result: OperationalError: database is locked
Connection 1: ROLLBACK complete
```

Connection 1 rolled back successfully. This confirms that the second connection could not acquire the writer reservation during this test. The experiment demonstrates writer serialization in the tested scenario; it does not demonstrate a committed write-skew anomaly.

### 6.4 Prevention and Trade-Off

The proposed prevention is to use `BEGIN IMMEDIATE` before reading and validating the invariant. The transaction reserves the writer slot before making its decision, preventing another writer from independently completing the same read-then-write sequence at the same time.

The observed test confirms that a second connection can be blocked from acquiring the writer reservation while the first holds it. A full demonstration that the same write-skew schedule is prevented and that the final application invariant is preserved would require running and recording the complete read, validation, update, and final-state sequence.

The trade-off is that competing writers must serialize, which can increase waiting time. If a connection cannot acquire the writer reservation within its timeout, it may receive `SQLITE_BUSY` and need to retry its transaction.

## 7 Conclusion

In this assignment, I examined the query plans of several ScrollSense queries on a scaled SQLite database. The index experiments changed the access paths used for F4, F7, and F12. In particular, SQLite used the signal-type index for the retraction query, the session date-range index for F7, and the persistent covering index for F12's correlated completion check.

I also compared textbook join-cost estimates. Under the simplified page model, block nested-loop join has a much larger estimated transfer cost than two-pass sort-merge or Grace hash join for the selected input sizes. These estimates are theoretical and do not represent measured SQLite costs, because SQLite's actual execution uses the join strategies available to it and the indexes in the database.

For predicate pushdown, I rewrote F7 so that the date filter is explicitly applied to `Session` before the joins. The captured execution plan already shows an indexed range search on `Session`, so the plan supports early filtering but does not establish an additional speedup from the rewrite.

Finally, the two-connection experiment demonstrated that `BEGIN IMMEDIATE` reserves the writer slot and prevents a second connection from acquiring it while the first holds it. The proposed write-skew schedule has a cyclic precedence graph, but a committed anomalous outcome was not observed. I have kept this distinction explicit in the report.

The three index experiments have recorded before-and-after wall-clock measurements: F4 took 0.09 seconds before and 0.03 seconds after indexing; F7 took 0.13 and 0.08 seconds; and F12 took 6.94 and 3.46 seconds. These are single-run observations, not controlled benchmarks, and may be affected by cache state and system load. A directly comparable timing experiment for the original F7 SQL and the explicit predicate-pushdown rewrite remains outstanding.

## Appendix: Reproducibility and Assistance

The working directory contains the SQL queries, schema, view definitions, index definitions, data-generation script, transaction demonstration, and analysis notes used in this work. The scaled database was kept separate from the original Assignment 1 files. The concurrency test was performed on a disposable database copy.

I used an AI assistant for help with understanding execution plans, structuring explanations, and editing the report. The final report should be read alongside the SQL files and the recorded experimental evidence. Any code, interpretation, and results submitted remain my responsibility to verify against the assignment requirements.